

Table of Contents

The Narrative Flow of the Project	1
Problem Statement of the Project	1
First Major Question of the Project -	3
Sub Question 1 -	4
Sub Question 2 -	6
Sub Question 3 -	8
Sub Question 4 -	10
Answer of the First Major Question -	12
Second Major Question of the Project -	14
Sub Question 1 -	15
Sub Question 2 -	18
Sub Question 3 -	19
Sub Question 4 -	22
Answer of the Second Major Question -	24
Third Major Question of the Project -	26
Sub Question 1 -	27
Sub Question 2 -	28
Sub Question 3 -	29
Sub Question 4 -	31
Answer of the third major question -	33
Fourth Major Question of the Project -	35
Sub Question 1 -	37
Sub Question 2 -	39

Sub Question 3 -	41
Sub Question 4 -	44
Answer of the fourth major question -	46
Answer to the Problem Statement.....	48
Summary of the Project.....	50
Key Findings.....	51
What Worked	52
What Did Not Work.....	53
Final Takeaway.....	54

The Narrative Flow of the Project

The project follows a problem-answer-architecture narrative, which is directly related to how the Scrum technique is used throughout the workflow. The project structure is divided into multiple layers so the overall system can remain manageable, modular, and easier to understand operationally.

The first layer is the problem statement itself, which acts as the overall theme of the complete project. This layer is not repeatedly highlighted inside every section, but it continuously drives the direction of the workflow. The second layer consists of the major questions of the project. These major questions work as the primary analytical stages that move the project forward operationally.

Each major question is then divided into four or five sub-questions. These sub-questions contain smaller operational tasks, analyses, or implementation checkpoints. Most of these tasks are saved as views inside the workflow, which I also refer to as checkpoints. Instead of building one extremely large and rigid structure in a single step, the Scrum technique allows me to work on smaller operational tasks individually so specific problems, dependencies, and analytical areas can be understood more clearly.

These views and checkpoints also become important for reusable continuity later on. Whenever new data is added or future refinements are needed, these checkpoints help validate the workflow without damaging or significantly affecting the complete downstream structure. Because of this, the project behaves more like a modular operational system where individual analytical areas can be checked, refined, or updated independently while still remaining connected to the overall architecture.

Overall, this layered structure helps the project remain modular, manageable, and operationally reusable. By dividing the workflow into smaller analytical checkpoints connected through views, the project becomes easier to validate, refine, and expand later on without disturbing the complete downstream structure.

Problem Statement of the Project

I have already completed two projects, one on machine learning and the other on natural language processing. While working on both projects, I continuously thought about how I had previously done many financial modelling tasks in MS Excel, which are mostly static in nature. Once the structure is created, future data usually requires manual updating, restructuring, and operational adjustments again. I did not want to go back toward that type of static workflow. Instead, I wanted to understand whether I could create something

more dynamic in nature, where future data could continuously enter the same structure with minimal effort.

This understanding became stronger when I started learning about SQL views. I realised that views can behave more like dynamic operational layers rather than static tables. If new data enters the same schema structure, the downstream values and analysis connected through the views can automatically update, similar to how a dynamic financial model behaves. At the same time, I also wanted the workflow to remain manageable and modular instead of becoming one extremely large and rigid structure. Because of this, I focused heavily on the Scrum technique, which complements the view-based workflow by dividing the project into smaller operational checkpoints.

This is where the foundation of this project emerged. In both my ML and NLP projects, I had already used Scrum-based structures with view-based checkpoints. In the ML Project specifically, the raw dataset was mainly used once as the source structure, and after that most of the downstream models, predictions, and analysis layers were connected through views. So this project mainly focuses on understanding whether that existing ML structure can behave like a reusable operational system when new data is continuously added using the same schema continuity.

For the NLP Project, however, the challenge is much more complex. Unlike BigQuery ML, which directly supports ML functions inside the SQL environment itself, BigQuery SQL does not directly support NLP extraction functions in the same reusable way. Because of this, the NLP workflow requires an external operational layer through Python and Jupyter Notebook for text extraction tasks. So another major focus of this project is understanding how this more modular and externally connected NLP workflow can also become more reusable when future data continuously enters the structure.

Another important objective of this project is understanding whether the overall analytical workflow can become more dynamic and less manual while still remaining manageable, modular, and operationally stable. Because of this, the project is not focused on building new ML or NLP models. Instead, the focus is on understanding whether the existing analytical architecture itself can behave like a reusable operational system where future market data and future text data can continuously flow through the same structure over time with controlled manual effort and stable downstream continuity.

Overall, the foundation of this project is not centered around building entirely new analytical models, but around understanding whether the existing ML and NLP workflows can behave like reusable operational systems. The project therefore focuses on how schema continuity, views, modular checkpoints, and reusable workflows can help future

data continuously move through the same analytical structure with controlled manual effort and stable downstream continuity.

First Major Question of the Project -

The first major question of this project focuses on understanding the structure that I have already created in both the ML and NLP Projects. I cannot simply add new data and check whether the workflows produce the expected outputs without errors unless I first understand how these project structures were originally built operationally. These projects are not limited to one or two models or analyses. Both workflows contain multiple analytical layers, checkpoints, views, and downstream connections that interact with each other throughout the system.

Because of this, the first thing I need to understand is how each analysis is structured, where the checkpoints are placed, which checkpoints are connected with each other, and how the outputs move from one stage of the workflow to another. Since these projects were built over a long period of time and divided into many smaller tasks, it is very common to lose visibility of the complete operational structure while focusing on individual analytical sections.

But if I want to test whether these workflows are actually reusable or not, then understanding the structure itself becomes the most important starting point. One important thing I also realised is that the Scrum technique, while extremely useful for dividing projects into manageable tasks, can sometimes make it harder to see how all the smaller operational parts connect together into one larger workflow. Some downstream dependencies or future operational connections can get overlooked while focusing only on isolated tasks. So understanding those hidden operational connections is one of the major purposes of this question.

Because of this, the first major question of the project is whether I can properly understand and map the operational structure of both the ML and NLP workflows in such a way that if future data creates an error, unexpected output, or operational inconsistency, I can identify where the issue is coming from and whether the overall workflow structure is truly reusable or not.

Overall, this first major question focuses on building a complete operational understanding of both workflows before testing their reusable continuity. Understanding how the checkpoints, dependencies, views, and downstream structures connect together becomes the foundation for analysing whether the existing ML and NLP architectures can actually behave like reusable analytical systems over time.

Sub Question 1 -

The first question or curiosity that comes to me while trying to understand the structure of these two projects is whether I can properly understand and reconnect what I have already built. Is this the right time to go back and reflect on the analyses, views, checkpoints, and models where I may not have gone deeply enough earlier? Normally, when a project gets completed with many different layers and tasks, people rarely go back and analyse the structure from the base level again. But when that same project is expected to behave like a reusable system in the future, understanding the base structure becomes extremely important. So the first major curiosity here is whether I am able to properly understand the foundational structure of both projects and how their different operational layers connect together.

At the same time, I also understand that this cannot be done in one step. It is not practical to look at the entire workflow at once and suddenly understand every dependency, connection, and operational layer together. The important thing is that while originally building these projects, I already had the intention that they should remain reusable in the future with minimal effort. This project is now testing whether that original intention was implemented properly or not. While working on the projects earlier, I did not fully think about which layers would later behave automatically and where manual operational work would still remain necessary. So understanding those operational distinctions becomes one of the main goals of this question.

As I started reviewing the structure more carefully, I noticed that I had already separated different major questions into different datasets. This separation itself helped create a more modular operational structure because each dataset performs a different responsibility while still remaining connected to the overall workflow. For example, in the ML Project, the classification model is one of the most important downstream structures. Because it exists as a separate analytical layer, I can later append new data, directly compare the updated classification outputs with previous outputs, and validate whether the model continuity still behaves similarly without disturbing the remaining ML workflow. This separation naturally makes the architecture more modular and operationally manageable.

A modular system basically means that different parts of the workflow perform different responsibilities while still remaining connected through a stable structure. This concept became very visible inside the ML workflow because BigQuery ML directly supports ML functions operationally inside the SQL environment itself. The raw data is provided once, and after that the processing, prediction flow, and analysis layers are connected mainly through views. Since these views dynamically update whenever new data enters the

same schema structure, most of the downstream analysis also updates automatically. Because of this, I do not need to repeatedly process, save, and rebuild the analytical layers again and again whenever future data enters the workflow. The interpretation layer still remains manual, like the analysis I am writing right now, but the downstream operational continuity itself behaves dynamically through the view-based structure.

Because of this understanding, the first implementation step is simply to review how all the datasets, views, and operational structures were originally created inside BigQuery SQL. Since each major question already exists as a separate dataset structure, reviewing them individually helps me understand where the workflow starts, where dependencies exist, how outputs move operationally, and where the workflow finally connects downstream. At this stage, the goal is not to modify anything, but simply to understand the complete operational picture of the architecture.

While reviewing the ML workflow more carefully, I understood that the raw input layer is mainly provided once, and after that the downstream models are built using views rather than directly using the raw dataset repeatedly. The first model itself, which is the regression model, is built from a view-based structure. The model can still require retraining later if there is major structural market change, performance degradation, or enough new market behavior accumulates over time. But under normal appended-data continuity, the future market data mainly enters as "Set_Type = test" while the model remains trained on the original training structure. Because of this, the reusable continuity of the ML workflow mainly exists in the downstream prediction and analysis layers rather than repeated retraining continuity.

Another important operational understanding here is that I have not used static downstream tables inside the ML workflow. Most downstream layers are dynamically connected through views. Because only the raw input changes while the downstream logic remains structurally connected, the analytical continuity updates operationally on its own without repeatedly rebuilding the complete workflow. This makes the ML architecture much smoother and more reusable operationally.

However, this operational smoothness changes significantly when the NLP workflow is analysed. Unlike BigQuery ML, BigQuery SQL does not directly support NLP extraction functions in the same reusable way. Because of this, the NLP workflow requires externally processed extraction outputs from Python before they are uploaded back into BigQuery SQL. The extracted outputs are stored first as tables and then used through downstream views for further analysis continuity. This means that whenever new NLP extraction outputs are generated through Python, the extraction tables themselves need to be refreshed or replaced again operationally.

This creates a major architectural difference between the ML and NLP workflows. In the ML workflow, once the raw input changes, the downstream structure updates smoothly as one connected dependency chain because the operational continuity exists almost fully inside the SQL environment itself. But in the NLP workflow, every major question behaves more independently because each signal depends on separately regenerated extraction tables. This means I can rerun or refresh one specific NLP signal independently without disturbing the complete NLP workflow. At the same time, some downstream analysis views still remain connected across datasets, so the NLP workflow still contains dependencies even though it behaves much more modularly than the ML structure due to the external Python extraction layer.

So operationally, understanding the ML structure feels smoother because the downstream continuity remains centrally connected through SQL views. The NLP workflow, however, becomes more complex because every major question behaves independently at the extraction level while still maintaining partial downstream dependencies inside the analysis layers. But this separation also creates flexibility, because specific NLP signals can be updated, rerun, or refined individually without disturbing the complete operational workflow.

Overall, this first sub-question helped me understand that both the ML and NLP projects were already designed with some level of reusable continuity in mind, but both workflows survive operationally in very different ways. The ML workflow behaves more like a centralized dependency-connected reusable system, while the NLP workflow behaves more like a modular reusable architecture with independent extraction layers connected through downstream analysis continuity. Understanding these structural differences becomes the foundation for testing how reusable both systems can actually become when future data continuously enters the workflow.

Sub Question 2 -

After understanding the operational structure of both the ML and NLP workflows, and how both systems function differently because BigQuery directly supports ML functions but does not support NLP extraction functions in the same reusable way, another important structural question comes forward. The question is which parts of these workflows behave automatically when new data enters, which parts are operationally reusable, and where manual intervention is still required repeatedly. Understanding this distinction becomes important because it helps identify both the strengths and limitations of the overall architecture.

At the same time, I also realised that the term “reusable system” itself can sometimes become misleading. It can create the impression that a completely reusable structure

means no manual work is required at all. But operationally, that is not realistic. No analytical system becomes fully automatic in every layer. Some operational parts remain reusable because they are internally connected through stable dependencies, while other layers still require manual triggering, monitoring, refreshing, or external execution depending on how the structure is built.

This distinction becomes especially visible in the NLP workflow because the NLP extraction layer exists outside BigQuery SQL inside Python and Jupyter Notebook. The Python environment mainly works as an external execution layer where the extraction logic is manually triggered whenever new text data needs processing. At the same time, the BigQuery SQL environment is structured much more dynamically through views and dependency-connected downstream analysis layers. Because of this operational separation, the ML workflow behaves much more smoothly internally, while the NLP workflow contains an additional external operational dependency layer.

So, the next important step is to clearly separate the workflow into two operational categories: the manual operational layers and the reusable operational layers.

The first category consists of the manual operational layers where repeated intervention or refreshing is still required. The biggest example of this is the Python execution environment used in the NLP workflow. Since BigQuery SQL does not directly perform NLP extraction functions in the same reusable way as BigQuery ML performs ML operations, the extraction process still needs to be executed externally. Another important manual layer is the process of collecting, appending, and preparing the raw datasets themselves. These datasets are usually managed through Python because it supports external libraries and external data handling much more flexibly. Along with this, the extracted NLP output tables uploaded into BigQuery SQL also need refreshing whenever newly processed NLP outputs are generated externally.

The second category consists of the reusable operational layers. These are mainly the internally connected SQL views, prediction layers, aggregations, and downstream analytical structures built inside BigQuery SQL itself. Once refreshed data becomes available inside the system, these dependency-connected views continue functioning dynamically without repeatedly rebuilding the analytical logic again and again. The downstream summaries, predictions, and analytical outputs automatically adjust through the dependency flow already present inside the architecture.

While understanding this operational distinction, I also realised something important about the manual Python execution layer itself. Even though Python requires manual execution, the operational burden is still reduced because the extraction logic can be converted into reusable script templates. The logic itself does not need to be rewritten repeatedly. In most cases, only the new incoming data changes while the extraction

workflow remains structurally similar. Because of this, the manual layer itself still contains a certain level of operational autonomy.

Another important thing I realised is that complete automation is not always operationally beneficial. Some level of manual control at the starting point of the workflow actually helps maintain awareness of what new information is entering the system and which operational layer is being refreshed. If everything becomes fully abstracted operationally, there is also a possibility of losing visibility of the workflow itself. So in some ways, this manual triggering process also helps maintain conscious operational awareness throughout the analytical structure.

Because of this, the position of the manual operational layers becomes very important. Most of the manual intervention exists at the beginning of the workflow, where new data is collected, prepared, or extracted, and at the end of the workflow, where interpretation and analytical understanding are written manually. In the middle of the architecture, however, the reusable continuity becomes much stronger because the downstream SQL views, aggregations, predictions, and analysis layers continue functioning dynamically once the structure is already established.

Overall, this sub-question helped me understand that the operational continuity of the project does not come from making every layer fully automatic. Instead, the reusable continuity mainly exists in the middle dependency-connected analytical layers, while the external execution layers and interpretation layers still remain consciously manual in nature. The ML workflow achieves this continuity much more smoothly because most operations remain internally connected within BigQuery SQL itself, whereas the NLP workflow behaves more like a semi-reusable modular architecture due to its dependency on external Python extraction layers.

Sub Question 3 -

After understanding the reusable and manual operational layers of both workflows, the next important thing that comes to mind is how the data itself moves through the overall architecture. Understanding the operational flow of data becomes extremely important because the reusable continuity of the system depends heavily on how smoothly the data travels from one layer to another.

If I take the example of the ML Project, the data mainly flows internally from one major-question dataset to another because most of the processing happens inside BigQuery SQL itself. Since the models, predictions, aggregations, and analysis layers are operationally connected within the same environment, the workflow does not face major external interruptions repeatedly. This is one of the main reasons why the ML workflow

behaves very close to a fully reusable operational structure. The views themselves become the primary operational channels through which the data continuously moves across the architecture.

The situation changes significantly in the NLP workflow because the data flow gets interrupted whenever a new major-question extraction process is required. In the NLP structure, the data first moves into the external Python environment, where the NLP extraction functions are executed. After that, the processed outputs are generated externally before they are uploaded back into BigQuery SQL. Because of this, the reusable continuity of the NLP workflow is strongly influenced by how the data reconnects across different execution environments. In the ML workflow, the continuity remains smoother because the data flows internally inside one operational ecosystem, whereas in the NLP workflow the operational continuity reconnects through bridge layers such as CSV extraction files and upload refresh stages.

This operational difference became much clearer when I reviewed the datasets and dependency structures of both projects more carefully. The ML workflow contains a simpler and more internally connected operational flow. The raw datasets first enter as input layers. Some preprocessing work still exists externally, such as Python-based market return generation and preprocessing, but compared to the NLP extraction structure, this external dependency remains much smoother operationally and is not the primary reusable challenge in this question.

Once the raw input enters the ML workflow, the processing, prediction generation, analysis layers, and downstream connections mainly happen inside BigQuery SQL itself. The outputs generated from one stage are stored as views and then used directly for further analysis and downstream dependency connections. Because of this, the workflow behaves like one continuously connected operational system where the input, processing, prediction, storage, and analysis layers remain internally synchronized through views and dependency-connected structures.

However, when I analysed the NLP workflow more carefully, I realised that the operational data flow contains many more transition layers. The raw text input first moves into Python, where NLP extraction functions process the data and generate structured extraction outputs. These outputs are then downloaded or stored as CSV files, which themselves become an additional operational bridge layer inside the workflow. After that, the CSV files are uploaded into BigQuery SQL, where they become the first operational storage layer for further downstream analysis continuity.

Because of this operational structure, each major question inside the NLP workflow behaves more independently at the extraction level compared to the ML workflow. The NLP structure therefore becomes more manual operationally because every extraction

stage requires separate execution and refresh continuity before reconnecting to BigQuery SQL again. At the same time, once the refreshed extraction outputs become available inside BigQuery SQL, the downstream views and analysis layers again return to behaving dynamically and operationally reusable.

This means that the NLP workflow contains more operational components, more transition stages, and more bridge layers compared to the ML workflow. The continuity still exists, but it survives through reconnection layers rather than one continuously flowing internal operational chain.

Another important thing I understood from analysing the data flow is that the data itself changes operational form while moving through the pipeline. Initially, the data starts as raw information. Then Python transforms it into structured extraction outputs. After that, CSV files behave as operational bridge layers connecting external extraction with internal orchestration. Finally, BigQuery SQL reconnects those outputs into reusable downstream analytical continuity through views and dependency-connected structures.

Because of this, the workflow is not simply about moving data from one stage to another. It is also about understanding how the data continuously transforms operationally while still maintaining downstream continuity across different environments and analytical layers.

Overall, this sub-question helped me understand that the reusable continuity of the overall architecture is deeply dependent on how operationally smooth the data flow remains inside the system. The ML workflow achieves stronger continuity because most of the processing, prediction, and analytical layers remain internally connected within the same operational environment. The NLP workflow, however, survives through bridge-connected operational continuity where external extraction, CSV transition layers, and BigQuery orchestration continuously reconnect together to maintain the reusable analytical structure.

Sub Question 4 -

After understanding the structure of both projects, identifying the reusable and manual operational layers, and analysing how the data flows throughout the workflows, the next important question becomes how this overall architecture will sustain itself when new data continuously enters the system. In simple terms, the main concern now is understanding what allows the existing structure to remain stable, reusable, and operationally functional over time. This naturally leads to another important question: what conditions or criteria must future data satisfy in order to survive properly within the existing workflow structure?

The main concept on which this entire reusable architecture depends is the schema, or what can also be understood as the fixed structural definition of the data. In very simple terms, the schema defines how the system recognises the incoming data. This includes the column names, field arrangements, datatypes, and the structural formatting through which the downstream operational layers understand and process the information. More than anything else, the reusable continuity of the project depends on the stability of these schemas.

This becomes important because the downstream views, dependencies, predictions, aggregations, and analysis layers are all connected operationally through these predefined structural rules. If the schema itself changes significantly, then the downstream operational continuity can become unstable, disconnected, or start generating errors depending on how deeply the dependencies are exposed throughout the pipeline.

This creates the most important implementation understanding of the reusable system. The raw input itself becomes the foundational schema definition for both the ML and NLP workflows. Because of this, whenever new data is added into the system, it must be appended in the same structural format as the original data. Otherwise, the schema continuity breaks, and the reusable architecture loses its operational stability. This means that reusability does not simply depend on adding “new” data. It depends on adding structurally compatible data that follows the same predefined schema continuity already established inside the workflow.

This understanding becomes much simpler in the ML workflow because the project mainly depends on one foundational dataset structure. Since most downstream operational continuity remains internally connected through views, prediction structures, and dependency-connected analysis layers, the reusable continuity mostly survives as long as the raw input schema remains stable. Once the input follows the same structural format, the downstream operational layers continue functioning dynamically through the internally connected SQL architecture.

However, the situation becomes much more complex inside the NLP workflow. Unlike the ML structure, where one primary operational schema largely drives the downstream continuity, the NLP workflow continuously introduces new operational schema layers across different major questions. Along with the shared market dataset also used in the ML workflow, the NLP architecture additionally contains separate news datasets, earnings-call datasets, and multiple extraction output structures generated through Python.

This means that the NLP workflow does not depend on only one schema continuity. Instead, every extraction table uploaded from Python into BigQuery SQL also contains its

own defined schema structure which must remain operationally consistent for the downstream reusable analysis layers to survive properly. Because of this, every major question inside the NLP workflow introduces its own schema dependencies, extraction continuity, upload refresh logic, and operational bridge structures that all need to remain stable together.

Another important thing I understood from this analysis is that the reusable continuity of the project is not actually dependent on the changing information itself. The information can continuously evolve over time as new market data, news data, and earnings data enter the system. What actually sustains the reusable structure is the stability of the operational framework through which the information moves. As long as the schemas, field names, datatypes, dependency structures, and downstream operational connections remain stable, the reusable continuity of the architecture can continue functioning even when the actual information inside the system changes continuously.

Because of this, the project is not simply testing whether future data can be added into the workflow. It is testing whether the operational structure itself is stable enough to continuously absorb changing information while maintaining downstream continuity without repeatedly rebuilding the complete analytical architecture again and again.

Overall, this sub-question helped me understand that the true foundation of the reusable system is not the changing data, but the stability of the schema-driven operational structure through which the data flows. The ML workflow achieves this more smoothly because its downstream continuity depends mainly on one internally connected schema structure, whereas the NLP workflow becomes more operationally sensitive because multiple datasets, extraction outputs, and bridge-connected schema layers all need to remain structurally consistent together for the reusable system to survive properly over time.

Answer of the First Major Question -

After going through the structure of both the ML and NLP projects step by step, I now understand that the real purpose of this first major question was not simply to check whether the workflows continue functioning when new data is added. The deeper purpose was to understand how the complete operational architecture is internally connected before even testing whether the structure is reusable or not. Without understanding where outputs originate from, how the views are connected, where dependencies exist, which layers update dynamically, and where manual intervention is still required, it would not be practical to simply append new data and expect the overall

workflow to behave correctly. So the main focus of this question gradually became understanding the internal operational continuity of the projects themselves.

After reviewing the structure carefully, I realised that both projects are already reusable to a significant extent, but both workflows survive operationally in very different ways because the underlying systems supporting them are different.

In the ML workflow, the structure behaves much more smoothly because most of the operational continuity exists completely inside BigQuery SQL itself. The raw dataset is mainly provided once, and after that the models, predictions, evaluations, aggregations, and downstream analysis layers remain connected primarily through views and internally connected dependency structures. Since the views update dynamically whenever structurally compatible new data enters the workflow, most of the downstream analytical continuity also updates automatically. Because of this, the ML workflow behaves as a much more reusable and less manually dependent operational structure.

One important operational understanding I gained here is that the model itself does not need retraining every time new market data enters the workflow. Most future market data mainly behaves as appended test-type continuity, while the model training itself already exists on the earlier training structure. Because of this, the reusable continuity of the ML workflow mainly survives in the downstream prediction, aggregation, and analytical structures built around the model rather than repeated retraining itself.

However, the NLP workflow behaves differently because BigQuery SQL does not directly support NLP extraction functions in the same reusable way as BigQuery ML supports ML operations. Because of this, the NLP workflow depends on the external Python environment for operational tasks such as sentiment extraction, entity extraction, and phrase-level processing. The processed outputs generated through Python are then uploaded back into BigQuery SQL, where the downstream analysis layers again become reusable through dependency-connected SQL views.

So compared to the ML workflow, the NLP architecture contains more manual operational layers because the Python extraction process needs to be rerun, the extraction outputs need refreshing, and the processed files need to be uploaded again whenever new data enters the workflow. Still, once the refreshed extraction outputs are uploaded with structurally consistent schemas, the downstream analysis layers again return to behaving dynamically and operationally reusable through the internally connected SQL architecture.

Another major understanding I gained from this analysis is that reusable continuity does not primarily depend on the changing data itself. Instead, it depends much more on the stability of the operational structure through which the data moves. Things such as column names, datatypes, schema continuity, dependency logic, and downstream view

connections must remain structurally stable. As long as those operational structures remain consistent, the downstream analytical continuity can continue functioning properly even when new information continuously enters the workflow. But if the schema changes or dependency continuity breaks, then the reusable operational structure can also fail.

I also understood much more clearly why I originally divided both projects into multiple datasets, checkpoints, views, and operational layers using the Scrum technique. Earlier, I mostly viewed those divisions as organisational project structures. But after analysing the workflows more carefully, I realised that these separations also help isolate operational problems inside the system. If a downstream issue, incorrect output, or dependency failure appears later, I can trace where the issue is occurring without disturbing the complete workflow architecture.

Another important understanding from this major question is that the reusable continuity of the architecture mainly survives in the middle operational layers of the workflow. The starting layer, where new data is collected, prepared, or externally processed, still requires some manual operational control. Similarly, the final interpretation layer also remains manually reflective in nature. But between these two ends, the internally connected views, aggregations, predictions, and downstream analytical structures behave much more dynamically and operationally reusable once the structure itself is properly established.

Overall, the answer to the first major question is yes, I am now able to properly understand and map the internal operational structure of both the ML and NLP workflows. I now understand how the workflows connect, where the reusable continuity exists, where manual operational intervention still remains, how the data moves across the architecture, and what structural conditions are necessary for the workflows to remain reusable when future data continuously enters the system. This understanding itself becomes the foundation for testing whether the complete analytical architecture can truly behave like a reusable operational system instead of remaining only a one-time static analytical project.

Second Major Question of the Project -

At the end of the first major question, the main concept I understood is that the schema acts as the backbone of the entire operational system. It is not only important for reusability, but also for the long-term sustainability of the workflow itself. If the schema becomes distorted, inconsistent, or structurally altered, then the downstream views, dependency-connected logic, models, validations, aggregations, and analysis layers can all become unstable or stop functioning properly. Because of this, understanding the

schema becomes the most important requirement for maintaining the reusable continuity of the architecture.

This also helped me realise that everything else inside the workflow is secondary to the structural stability of the schema itself. The views, models, validations, predictions, and downstream analysis layers are all operationally connected through the fixed structural definitions already established inside the system. If those structural definitions remain stable, then the downstream operational continuity can survive even when the information inside the system continuously changes over time.

Because of this understanding, the second major question of the project focuses on the structure of the future data that will later be appended into the workflow. The question is what the schema or fixed structure of the future data should look like, how the new data should be structured so it does not disturb the existing operational continuity, and what validation rules are necessary to ensure that the reusable architecture remains stable when future data continuously enters the system.

Another important thing I understand here is that reusability is not only dependent on whether new data can technically enter the workflow. It also depends on whether the incoming data behaves structurally compatible with the operational logic already present inside the architecture. Even small inconsistencies in column names, datatypes, formatting structures, dependency connections, or extraction outputs can create downstream instability if proper validation continuity does not exist.

Because of this, the second major question is not simply about adding future data into the system. It is more deeply connected with understanding how the operational structure itself can continuously absorb future information while preserving schema stability, dependency continuity, and downstream analytical reliability across both the ML and NLP workflows.

Overall, this major question shifts the focus from understanding the existing reusable structure toward understanding the conditions required for that reusable structure to survive properly in the future. The core objective now becomes identifying the schema consistency rules, validation structures, and operational safeguards necessary to ensure that the analytical architecture remains stable, reusable, and operationally sustainable when future data continuously enters the workflow over time.

Sub Question 1 -

The first thing I need to understand before collecting or appending any future data is what type of schema structure the existing datasets already accept. Before bringing new

information into the workflow, it becomes necessary to first understand the current schema continuity on which the existing projects are already functioning. Only after understanding that structure properly can future data be collected and appended in a way that remains operationally compatible with the reusable architecture.

While reviewing the schemas again, I realised that schema continuity is not only about the placement of columns or the overall layout of the dataset. The datatype structure inside those columns becomes even more important because the datatype defines what type of information the system operationally expects. This is also where many downstream errors usually originate. If the expected datatypes, naming structures, or field continuity become inconsistent, then the downstream operational layers connected through views and dependencies can become unstable.

One important thing I also realised is that reusable systems can sometimes become more vulnerable to schema instability compared to more manually separated systems. In the ML workflow, most of the operational continuity remains smoothly connected internally through views, prediction layers, and dependency-connected structures. Because of this smoothness, even small structural schema disruptions can affect the downstream workflow much more quickly. Since the architecture is highly connected operationally, a schema inconsistency can propagate across multiple downstream analytical layers very easily.

The NLP workflow behaves differently in this situation because each major question remains more operationally separated due to the external Python extraction layer. This separation actually reduces some level of structural vulnerability. For example, if one extraction schema encounters an issue but another major-question dataset remains structurally stable, then a specific downstream analysis can still continue independently. If I only need the theme analysis layer and its schema continuity remains correct, then I can still continue using that particular operational structure even if another NLP extraction layer encounters problems. This level of separation does not exist in the ML workflow because most downstream structures remain much more centrally connected.

So one important understanding from this analysis is that the operational smoothness of the ML workflow also comes with higher dependency exposure and greater schema sensitivity. The NLP workflow is operationally more fragmented, but that fragmentation also creates some degree of isolation flexibility when schema-related issues occur.

Another important thing I understood while reviewing the structures is that reusable systems survive operationally because the structure remains predictable, not because the future information itself remains fixed. Market conditions, news events, earnings narratives, and future financial information can continuously change over time. But as long as the schema continuity, datatypes, naming structures, append patterns, and

dependency logic remain stable, the downstream reusable continuity can continue functioning operationally.

While inspecting the ML project schema more carefully, I also realised that the dataset currently contains many columns that are not actively being used in the downstream workflow. For example, columns related to OHLC daily pricing structures such as open, high, low, and close prices are present, even though the actual operational workflow mainly depends on return values and Nifty return structures. These additional columns are not entirely unnecessary because they can support future project expansion later on. However, operationally they still increase the workload of future data collection because the structure expects information that is not directly contributing to the current downstream analytical continuity.

This made me realise that datasets can also accumulate unused operational layers over time, similar to how physical storage spaces slowly accumulate items that are rarely used actively. In analytical systems, this can be understood as a form of over-collection or underutilisation of data resources where additional information exists structurally but does not meaningfully contribute to the active downstream workflow.

Interestingly, this situation does not exist as strongly in the NLP workflow. The NLP datasets are much smaller structurally and usually contain only a few operationally necessary columns. Most of those fields are actively contributing to the downstream extraction and analysis layers. Because of this, the data collection process in the NLP workflow is operationally more focused and efficient since fewer unnecessary fields need to be maintained continuously.

This also highlights another structural difference between the two projects. The NLP workflow mainly focuses on operationally necessary fields, whereas the ML workflow currently contains a mixture of actively used operational fields and future-expansion-oriented fields together within the same schema structure.

Overall, this sub-question helped me understand that schema continuity is not only about preserving structural compatibility for reusable workflows, but also about understanding how efficiently the schema itself supports the operational architecture. The ML workflow achieves smoother downstream continuity, but it also becomes more structurally sensitive because of its highly connected dependency structure. The NLP workflow, while operationally more fragmented, remains more focused toward necessary operational fields and therefore behaves with greater schema isolation flexibility in some analytical situations.

Sub Question 2 -

After understanding the schema structures of both projects, the next important thing is understanding how future appended data should be structured so it does not create instability inside the reusable workflows. This becomes especially important for the ML project because its downstream continuity is much more dependency-connected compared to the NLP workflow. In the NLP structure, each new major question already behaves more independently due to the external Python extraction layer, so operational adjustments can still be managed more flexibly whenever a specific analysis needs modification.

In practical terms, the most important operational structures for me are the classification model in the ML project and the theme signal in the NLP project. However, both behave very differently operationally. In the ML workflow, the classification model cannot remain stable if the connected upstream and downstream structures become inconsistent, because most models and analysis layers remain highly connected through the same internally shared dependency structure. In the NLP workflow, the theme signal behaves much more independently because every major-question extraction layer is more operationally separated from the others.

At the same time, the NLP workflow is still partially dependent on the market return dataset because some downstream analytical comparisons continue using shared financial continuity. But compared to the ML workflow, the operational dependency exposure remains much lower. This means that partial analysis continuity can still survive operationally in the NLP structure even when another signal layer encounters structural issues.

Because of this, compatibility becomes much more important operationally in the ML workflow. Simply matching column names is not enough for future appended data to behave correctly inside the reusable system. The datatype continuity, formatting structure, naming consistency, append patterns, and overall schema compatibility all become equally important because the downstream operational layers continuously depend on the same shared structural foundation.

One important thing I realised here is that even very small schema mismatches can create major instability inside the ML workflow. A datatype mismatch, formatting inconsistency, or structural incompatibility can affect the downstream operational continuity very aggressively because the ML architecture is highly centralized around the same shared return-based structure. Most downstream models continuously reuse the same foundational columns such as stock returns and index return structures. Because of this shared dependency exposure, even a small incompatibility can propagate operationally throughout multiple downstream layers very quickly.

The NLP workflow behaves differently because the extraction structures remain more modular and partially isolated operationally. Even though some downstream analytical continuity still connects through market return structures, the external extraction separation reduces the degree of centralized dependency exposure. Since fewer columns are used operationally and each major question behaves more independently, structural incompatibilities remain more isolated instead of spreading aggressively across the complete workflow.

So while the ML workflow achieves smoother reusable continuity through strong internal connections, that same smoothness also creates much higher structural vulnerability. The downstream operational layers become extremely dependent on the stability of the top-level schema continuity. The NLP workflow, on the other hand, behaves in a more modular operational manner where specific signal layers can continue functioning more independently even if another extraction layer becomes unstable.

Another important thing I understood from this analysis is that the strength of a reusable system and its vulnerability often come from the same source. In the ML workflow, the tightly connected dependency structure creates excellent downstream continuity when the schema remains stable, but the same interconnectedness also increases the operational risk whenever incompatibilities enter the structure. The NLP workflow sacrifices some level of smooth continuity because of its external bridge layers, but that operational fragmentation also creates greater isolation flexibility when structural problems occur.

Overall, this sub-question helped me understand that future appended data must not only look structurally similar on the surface, but must remain deeply compatible with the dependency-connected operational logic already present inside the workflows. This compatibility becomes especially critical for the ML project because its reusable continuity depends heavily on centralized schema stability, whereas the NLP workflow survives operationally through a more modular and partially isolated structure.

Sub Question 3 -

After understanding the compatibility requirements of the future data and the structural vulnerability present inside the ML workflow, another important understanding comes forward. Since the ML architecture is highly interconnected through shared dependencies, even small incompatibilities can make the downstream continuity operationally unstable. Because every model and downstream analysis layer depends on the same foundational raw input structure, it can become difficult to identify where the actual incompatibility is originating once instability spreads across the workflow.

At the same time, I also realised something important about this dependency structure. Since most downstream models are connected through the same raw input continuity, if the dataset remains structurally compatible for one downstream model, it will usually remain compatible for the other downstream layers as well. This creates a type of shared operational continuity across the ML workflow where stability or instability tends to propagate collectively through the same dependency chain.

Because of this, the top-level raw input layer becomes the most critical operational point inside the ML architecture. Protecting the raw input continuity is not only important for future reusability, but also for protecting the stability of the existing downstream workflow itself. Inside this raw input layer, the schema remains the most important structural component that needs to stay consistent operationally.

While reviewing the structure more carefully, I also realised that even though the ML dataset contains many additional columns, the actual downstream operational continuity mainly depends on a much smaller group of consistently used fields. The stock return columns, index return columns, Set_Type column, and Date column remain continuously used throughout almost the entire workflow. Because these core fields remain structurally stable across downstream models and analysis layers, the overall operational dependency structure becomes easier to understand and maintain.

However, the NLP workflow behaves very differently operationally. In the first few major questions, the market return continuity mainly depends only on the index return column, while later stages additionally include stock return continuity as well. But unlike the ML workflow, the NLP major questions remain much more operationally independent because every extraction layer reconnects externally through Python processing. Because of this, the NLP workflow behaves as a very different type of reusable system compared to the ML architecture.

This separation reduces the level of global instability created by schema incompatibilities inside the NLP workflow. Since the extraction layers remain more isolated operationally, one unstable major-question dataset does not necessarily damage the entire downstream structure. At the same time, the NLP workflow introduces another important challenge: the external extraction continuity itself also needs to become reusable.

The news datasets and earnings datasets used inside the NLP workflow must not only remain structurally compatible inside BigQuery SQL, but they must also remain operationally compatible with the Python extraction scripts used for generating the NLP outputs. Since the same extraction logic will later process newly appended data, the external extraction layer itself also needs reusable continuity with minimal future modifications.

This creates another operational challenge that becomes important for the long-term sustainability of the NLP workflow. The goal is not simply to make the downstream SQL analysis reusable, but also to minimise the operational effort required to repeatedly process and extract future NLP signals externally through Python.

Because of this understanding, another major focus now becomes creating a reusable framework for how the raw input data itself should be collected operationally in the future. Until now, most of the analysis started after the raw input already existed, but long-term reusable continuity also depends heavily on how consistently the raw input itself is collected and prepared over time.

In the ML workflow, the raw input data is collected using Python libraries that extract historical market data for the index and selected stocks. Because of this, the reusable continuity can be improved significantly by creating a reusable Python template where only the date ranges need adjustment whenever future updates are required. Since the template already contains nearly the same schema structure used in the ML workflow, most of the operational continuity can remain structurally stable with minimal future changes.

The template mainly adds the return columns and Set_Type continuity required for the downstream ML workflow, while the remaining schema structure already stays operationally consistent. This becomes especially important because the ML dataset currently contains several additional columns that are not actively used operationally, such as OHLC pricing columns. So using the same reusable collection template helps maintain schema continuity more reliably across future updates.

The NLP workflow, however, becomes much more challenging from a data collection perspective. Unlike historical market data, which is relatively easier to retrieve consistently, news data behaves much more differently operationally because of its time-sensitive and disappearing nature. One of the biggest limitations I faced during the NLP project was the scarcity and availability of older news data itself.

While recent news data can usually be collected in large quantities, older news gradually becomes difficult to retrieve properly over time. This became very visible while trying to collect historical news continuity across long periods during the NLP project. Because of this, future reusable continuity in the NLP workflow will require a much more systematic and prioritised framework for collecting and preserving news data continuously before it becomes unavailable.

Similarly, the earnings data extraction process also requires a more stable operational framework for future continuity. Because of this, the focus now shifts toward building reusable ingestion frameworks that support repeated future updates instead of supporting only one-time project execution.

Another important understanding I gained here is that these reusable frameworks themselves will continue evolving gradually over time. Not every operational challenge becomes visible during the first implementation itself. As future updates happen and new limitations appear, the ingestion frameworks will also need gradual refinement and operational improvement.

So operationally, the focus now is not only on making the existing workflows reusable once, but on creating stable collection and ingestion frameworks that allow the reusable systems to survive continuously over multiple future updates with controlled manual effort and stable schema continuity.

Overall, this sub-question helped me understand that long-term reusable continuity depends not only on downstream analytical structures, but also on how consistently the raw input itself is collected, processed, and ingested into the workflow over time. The ML workflow mainly depends on maintaining stable centralized raw input continuity, whereas the NLP workflow additionally depends on building reusable external ingestion and extraction frameworks capable of supporting future operational continuity across changing datasets and evolving information sources.

Sub Question 4 -

While working on the raw input data framework and finalising how future data will be collected and appended, another important operational question comes forward: why are checks and balances necessary, and what are they actually protecting inside the reusable system? Without validating the framework properly at different stages, the overall workflow can become much more vulnerable to operational instability and downstream errors. Because of this, validation checks become important for protecting the reusable continuity of the architecture itself.

These checks and balances mainly exist in the form of validation checks. Operationally, validation means confirming that the extracted or appended data remains structurally compatible with the existing workflow. This includes checking whether the datatypes remain consistent, whether the column structures match correctly, whether the naming continuity remains stable, and whether the appended data behaves operationally compatible with the existing downstream dependency structure.

Another important thing I realised is that these validation checks should not only happen after the data enters the workflow. The framework itself should already be designed in a way that minimizes structural inconsistencies from the beginning. If the collection template or ingestion framework itself continuously produces structurally inconsistent

outputs, then the downstream reusable continuity will remain unstable no matter how strong the analytical layers are later.

This becomes especially important in the ML workflow because the dataset contains a large number of columns and continuously expanding historical market rows over time. Since the downstream models, predictions, aggregations, and analysis layers remain highly centralized around the same dependency structure, even relatively small schema inconsistencies can create operational instability very quickly. As the dataset grows larger over time, validating datatype continuity, column consistency, and structural compatibility becomes increasingly important for maintaining downstream operational trust.

The NLP workflow behaves somewhat differently operationally because the datasets are structurally smaller and contain fewer fields. Most NLP datasets mainly contain a small number of columns, and apart from the Date column, many fields are primarily text-based string structures. Because of this, the NLP workflow does not face the same level of centralized schema sensitivity that exists in the ML architecture.

However, this does not mean validation becomes unimportant in the NLP workflow. The NLP structure introduces a different category of operational risks. Problems such as extraction inconsistencies, empty text outputs, formatting irregularities, incomplete extraction continuity, or unstable processed outputs can still affect the downstream analytical reliability of the workflow. So even though the NLP workflow is less structurally centralized compared to the ML architecture, validation continuity still remains important operationally.

At this stage, I am still understanding and refining how the data frameworks for both projects should ultimately behave operationally. The next major step after this will be to actually append newly extracted data using these ingestion frameworks and observe how the reusable workflows behave with the updated information operationally.

One important thing I understand now is that validation checks are not simply technical formalities inside the workflow. Their main purpose is to protect the stability of the downstream reusable continuity. In the ML workflow especially, the centralized dependency structure makes validation much more critical because operational trust across the complete downstream architecture depends heavily on the structural stability of the raw input itself.

Overall, this sub-question helped me understand that checks and balances are necessary because reusable systems survive only when structural continuity remains protected consistently over time. Validation checks therefore become one of the main operational safeguards that help maintain schema stability, downstream reliability, and reusable continuity across both the ML and NLP workflows, even though the ML project

remains much more structurally sensitive due to its highly centralized dependency-connected architecture.

Answer of the Second Major Question -

After going through all the sub-questions of this second major question, the main thing I understand is that the sustainability and reusable continuity of the entire architecture depend primarily on schema stability and the compatibility of future appended data with the existing operational structure. The schema is not simply a collection of columns. It acts as the operational backbone through which the downstream views, models, validations, dependency structures, and reusable continuity survive. If the schema continuity weakens, then the downstream operational synchronization and reusable analytical flow also become unstable.

Because of this, the most important thing before appending future data is understanding what type of structure the existing workflow operationally accepts and what datatype continuity, formatting consistency, naming structure, and append logic the downstream system continuously depends on. The future information itself does not need to remain fixed because market behavior, earnings narratives, and news continuity will naturally keep changing over time. But as long as the schema continuity, datatype structure, and dependency-connected append continuity remain stable, the reusable architecture can still survive operationally.

Another major understanding I gained is that the ML and NLP workflows behave very differently in terms of operational compatibility and dependency exposure. The ML workflow behaves as a highly centralized and downstream-dependent structure where most models continuously depend on the same foundational raw input continuity, especially the stock return columns, index return column, Set_Type column, and Date continuity. Since the same structural foundation gets reused across the downstream workflow repeatedly, even a small incompatibility in datatype or schema continuity can propagate aggressively throughout the complete operational architecture.

The NLP workflow behaves much more modularly because the major-question structures remain operationally separated through external extraction continuity. Even though some downstream analytical continuity still depends on the market return structure, especially the index return continuity, the overall NLP workflow remains comparatively less globally vulnerable than the ML architecture. The NLP datasets also remain more operationally focused with fewer actively used fields, whereas the ML workflow currently contains both actively used operational columns and future-expansion-oriented fields together within the same schema continuity.

Another important understanding from this major question is that the raw input layer becomes the most critical operational point inside the reusable architecture. Since the downstream workflows continuously depend on the raw input continuity, protecting the schema structure and datatype stability of the raw input becomes extremely important for preserving reusable continuity across the system.

This also helped me understand that future reusability does not depend only on appending new data once. It depends much more on creating reusable ingestion frameworks capable of continuously collecting and integrating structurally compatible future data into the workflow over time.

For the ML workflow, this reusable continuity mainly depends on reusable Python-based extraction templates that can continuously collect historical market data while preserving the same schema continuity already used in the project. Operationally, most of the structure remains stable while mainly the date continuity changes during future updates.

The NLP workflow becomes more operationally challenging because news continuity and earnings continuity are not historically available in the same stable way as market return data. News data especially behaves as a time-sensitive operational resource because older news gradually becomes more difficult to retrieve properly over time. Because of this, creating reusable collection and extraction frameworks for continuously preserving news continuity and earnings continuity becomes operationally important for the long-term sustainability of the NLP architecture.

Another important thing I understood is that creating reusable ingestion frameworks alone is not sufficient for maintaining operational continuity. Reusable systems also require validation-governed protection layers so future extracted continuity does not weaken the downstream workflow structurally.

These validation checks mainly focus on datatype continuity, column continuity, formatting consistency, schema compatibility, and downstream operational synchronization with the existing reusable architecture. This becomes much more important in the ML workflow because of its highly centralized dependency structure, large dataset continuity, and strong downstream synchronization exposure. Even small validation failures can weaken the operational continuity aggressively there.

The NLP workflow remains comparatively less structurally sensitive because of its more modular architecture and smaller operational datasets. However, validation continuity still remains important operationally because extraction consistency, text formatting continuity, empty structures, and processed output reliability still affect downstream analytical quality.

Overall, the answer to the second major question is that reusable systems survive operationally not because future data always remains clean or fixed, but because schema continuity, ingestion frameworks, append compatibility, and validation structures continuously protect the downstream operational trust of the architecture over time. The ML workflow achieves smoother reusable continuity but becomes much more structurally vulnerable because of its centralized dependency exposure, whereas the NLP workflow behaves more modularly with greater operational flexibility but higher external extraction dependency.

Third Major Question of the Project -

Now that I have understood the structure of both projects and the importance of schema continuity for maintaining reusable workflows, the next step is to apply this understanding directly to the ML system operationally. The main objective of this question is to build a reusable framework for collecting the raw input data of the ML workflow and then append the new compatible data into the existing structure.

The key thing I now want to understand is whether the ML workflow can behave like a reusable inference-based system when future data enters the same fixed schema continuity. Since the workflow is highly dependency-connected through views, models, predictions, and downstream analysis layers, even a small inconsistency in the source input or schema continuity can affect the downstream outputs operationally.

Another important part of this question is understanding whether the downstream views, prediction flow, and analysis continuity automatically update correctly after the new data is appended without requiring repeated retraining of the models. Since the reusable continuity of the ML workflow mainly survives through the downstream operational chain connected around the models, testing how this chain behaves with newly appended data becomes the main focus of this stage.

This question therefore shifts the project from structural understanding toward practical operational testing of the ML reusable system. The main goal is to observe whether the existing ML architecture can absorb future compatible data smoothly while maintaining downstream continuity, prediction synchronization, and reusable operational stability without rebuilding the complete workflow again.

Sub Question 1 -

Now the main focus of this project is to build a reusable framework for the raw input data used in the ML Project. The purpose of this framework is not only to support the current update, but also to make future data collection more reusable and operationally stable over time.

The first thing I already understand while building this framework is that the schema continuity of the framework should remain similar to the existing raw input dataset. The template itself should therefore be built while keeping the existing schema structure in mind, including the same column continuity already used in the ML workflow. This becomes important because the downstream views, prediction layers, and analysis continuity of the ML Project already depend on that existing schema structure. If the schema continuity changes, then the downstream operational continuity can also become unstable.

Another important thing is that the quality and source continuity of the future data should remain consistent with the existing dataset. If the source changes later, then the extracted data can also behave differently operationally, which may create issues such as datatype inconsistencies, missing fields, numerical mismatches, formatting changes, or different market conventions. Since the ML workflow is highly dependency-connected, even small inconsistencies in the raw input layer can propagate across the downstream inference structure.

Because of this, the framework should use a reusable template with a fixed extraction source and predefined schema continuity matching the existing ML dataset structure. The framework should therefore be built using the same Python extraction logic that was originally used for collecting the earlier raw input data, but structured in a more reusable and operationally dynamic manner instead of behaving like a one-time extraction setup.

So the next step is to implement the reusable framework itself. The extraction source will remain fixed through the same Python library used earlier so future data continuity remains operationally consistent. For the market return data, the framework uses the Yahoo Finance Python library along with the stock and index tickers already used in the ML Project. At the same time, the ticker structure itself can remain expandable later if additional assets need to be added into the workflow in the future.

Another important thing while designing this framework is making the starting and ending dates reusable operational parameters instead of hardcoded values. This allows future data extraction to happen by simply changing the date continuity without modifying the overall extraction logic itself. Because of this, the framework behaves more like a reusable operational module instead of requiring repeated manual rebuilding.

One important understanding I gained while working on this framework is that reusable systems are not only about automation. The real objective is to create a controlled and repeatable operational structure where future compatible data can continuously enter the same workflow without disturbing the downstream inference continuity already present inside the architecture.

Overall, this raw input framework becomes important because it helps preserve schema continuity, source consistency, operational stability, downstream compatibility, and reusable inference continuity across the ML workflow whenever future data needs to be appended into the existing system.

Sub Question 2 -

After building the reusable framework for extracting the raw input data, the next step is to merge the individual stock CSV files into one centralized dataset structure for the ML workflow. I decided to first extract separate CSV files for each stock instead of directly combining everything together in one step. This makes the workflow easier to manage operationally, reduces the possibility of extraction errors, and allows the data cleaning process to remain more modular and understandable.

Another important advantage of keeping the stock files separate initially is future expandability. If additional stocks are introduced later, it becomes easier to decide whether those assets should be included in the centralized ML structure or not. This keeps the raw input workflow more flexible operationally.

Now the main operational question becomes how these separate stock CSV files should be merged together into one centralized dataset structure.

One possible way is manual merging, where one CSV file is opened and the remaining stock data is copied and combined manually. While this may work when the number of files is small, it still creates operational risks. One of the biggest issues is alignment continuity. If one stock does not contain return continuity for a particular date while other stocks do, then the row synchronization can become mismatched during manual merging. Since the ML workflow is highly dependency-connected, even small alignment inconsistencies inside the centralized dataset can later affect the downstream inference continuity and prediction structure.

Because of this, using Python for the merging process becomes operationally more reliable and reusable. Instead of repeatedly rebuilding the centralized dataset manually whenever future market data is added, the merge workflow itself can behave like another reusable operational module inside the ML architecture.

While implementing this merging framework, some important reusable operational components were added into the Python structure. One of them is the glob library, which helps automatically identify files using common naming patterns. Since all the extracted stock files follow the same schema continuity and naming structure, the system can automatically detect and process them together without manually selecting every file repeatedly. This makes the workflow more scalable if more stocks are added later.

Another important operational component is the reduce function, which sequentially merges multiple datasets into one centralized dataframe structure. During this merging process, the Date column is used as the common operational key, which helps preserve synchronized market continuity across all assets together and reduces the alignment problems that could occur during manual merging.

The column structures are also renamed using ticker-based identities so the downstream ML workflow can still recognize which columns belong to which stock after the centralized dataset is created. This helps preserve feature identity continuity throughout the reusable inference structure.

After implementing the merging process, the final merged dataset now behaves like one centralized market-wide operational structure instead of disconnected stock files. This creates a much more reusable ML-ready dataset continuity for future downstream inference workflows.

Overall, this stage helped transform the raw extracted stock files into a synchronized and reusable centralized market dataset structure. The workflow now behaves more like a scalable operational merging framework instead of a manually rebuilt process, which improves downstream continuity, schema synchronization, and future reusable stability for the ML Project.

Sub Question 3 -

After merging the individual stock files into one centralized dataset structure, there are still two important preprocessing steps that need to be completed before uploading the updated data into BigQuery SQL.

The first step is to add some operational columns manually. These columns can also be generated using Python, but I intentionally want some parts of the preprocessing continuity to remain under my supervision and operational control. While adding these columns manually, I can also skim through the dataset, inspect the continuity, and check for null or NA values, which are common in financial datasets. This helps me understand

the data quality and operational stability better before the downstream inference workflow begins using the updated structure.

The first major preprocessing step is creating the return columns using the `ln` function in Excel for generating continuous returns. These return-based columns are important because the ML workflow already depends on return continuity rather than only raw price continuity. The downstream inference structure was originally built using these operational return layers, so the appended data must also follow the same continuity.

The second important step is setting the `Set_Type` column as test for all the newly extracted data from November 2025 to April 2026. This becomes critical for maintaining reusable inference continuity without retraining the models again. The new appended data cannot be marked as train because future information should not enter the training continuity operationally. Doing that could create data leakage, distort downstream evaluation continuity, and weaken the operational reliability of the inference structure itself.

Once these preprocessing steps are completed, the new data is appended into the original dataset continuity. After that, the updated dataset is uploaded into BigQuery SQL by replacing the existing raw input table using the same table name already connected throughout the workflow.

This becomes important because the downstream views, dependency structures, prediction continuity, and reusable inference layers are already operationally connected to that existing table structure. By preserving the same schema continuity, table continuity, and naming continuity, the downstream operational flow should ideally reconnect automatically with the newly appended data without requiring structural rebuilding.

At this stage, the reusable continuity of the ML workflow now moves from framework preparation toward actual operational validation. The preprocessing structure has been completed, the new compatible data has been appended into the existing continuity, and the updated dataset is ready to reconnect with the downstream inference workflow.

Now this becomes the real operational test of the complete understanding developed throughout the earlier major questions. The main question is whether the downstream reusable structure will continue functioning smoothly after replacing the existing raw input with the updated dataset, or whether hidden dependency issues, operational inconsistencies, schema mismatches, or downstream synchronization problems still exist somewhere inside the workflow architecture.

continuity, reusable framework, ingestion structure, and downstream operational flow theoretically. But now the reusable system itself will be tested practically by replacing the existing raw input structure with the newly appended dataset.

One important thing I understood about the ML workflow is that I do not need to validate every individual downstream model separately to understand whether the reusable continuity is functioning correctly. Since the ML Project behaves as a highly interconnected dependency structure, most of the downstream operational continuity depends on the same schema foundation, feature continuity, and dependency-connected architecture. Because of this, validating one major downstream inference structure carefully can already provide a strong indication of whether the overall workflow continuity remains operationally stable.

The implementation therefore started by first reviewing the existing downstream analysis continuity carefully. The total number of rows in the existing workflow was 2467, and after appending the new market data, the expected continuity should increase to 2586 rows. This became one of the first operational validation checkpoints for confirming whether the new data had propagated properly throughout the downstream ML structure.

At this stage, the focus of the project is not on analysing why prediction outputs changed or how the model behavior shifted after adding the new rows. The purpose here is much more operational in nature. The objective is to understand whether the reusable inference continuity of the existing ML architecture survives correctly when future compatible data enters the workflow.

The implementation then moved toward replacing the original raw input table inside BigQuery SQL. Before uploading the updated dataset, the existing raw input table was removed so the new appended dataset could replace it using the same table name already connected throughout the workflow. Since the downstream views, dependencies, prediction continuity, and analysis layers were already operationally connected to that table name, preserving the same naming continuity became important for maintaining reusable synchronization across the system.

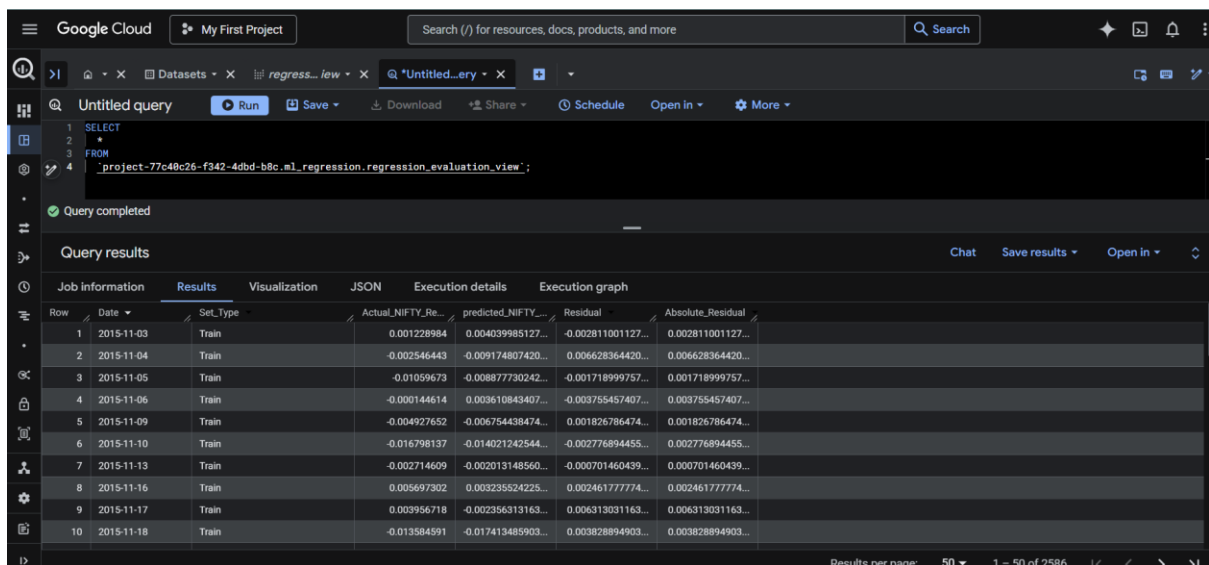
After uploading the updated dataset, the downstream analysis continuity was checked again. The total row count updated successfully from 2467 to 2586, and no schema issues, dependency failures, or downstream operational errors appeared after the replacement. This became one of the strongest operational validations of the reusable ML workflow because the complete downstream inference continuity successfully reconnected with the newly appended data on the first attempt itself.

This also confirmed that the broader operational understanding developed throughout the earlier major questions was largely correct. The schema continuity, reusable ingestion framework, downstream dependency synchronization, and operational

architecture of the ML workflow all behaved as expected when future compatible data entered the same reusable structure.

Another important thing I realised from this implementation is that the ML Project workflow genuinely behaves like a smooth dependency-connected reusable inference system when the schema continuity and source structure remain stable. The operational continuity survives not because the downstream workflow is manually rebuilt repeatedly, but because the dependency-connected views and analysis structures automatically reconnect once the compatible raw input continuity is preserved correctly.

Overall, this sub-question became the first real practical validation of the reusable ML architecture. The downstream inference continuity, schema synchronization, and operational dependency flow all remained stable after replacing the raw input structure with the newly appended dataset. With this validation now complete, the focus of the project can gradually move toward the NLP workflow, which operationally behaves much more differently and will likely become the more challenging reusable system to validate and stabilize.



Row	Date	Set_Type	Actual_NIFTY_Re...	predicted_NIFTY...	Residual	Absolute_Residual
1	2015-11-03	Train	0.001228984	0.004039985127...	-0.002811001127...	0.002811001127...
2	2015-11-04	Train	-0.002546443	-0.009174807420...	0.006628364420...	0.006628364420...
3	2015-11-05	Train	-0.01059673	-0.008877730242...	-0.001718999757...	0.001718999757...
4	2015-11-06	Train	-0.000144614	0.003610843407...	-0.003755457407...	0.003755457407...
5	2015-11-09	Train	-0.004927652	-0.006754438474...	0.001826786474...	0.001826786474...
6	2015-11-10	Train	-0.016798137	-0.014021242544...	-0.002776894455...	0.002776894455...
7	2015-11-13	Train	-0.002714609	-0.002013148560...	-0.000701460439...	0.000701460439...
8	2015-11-16	Train	0.005697302	0.003235524225...	0.002461777774...	0.002461777774...
9	2015-11-17	Train	0.003956718	-0.002356313163...	0.006313031163...	0.006313031163...
10	2015-11-18	Train	-0.013584591	-0.017413485903...	0.003828894903...	0.003828894903...

Answer of the third major question -

After understanding the structure of the ML workflow, the schema continuity, and the dependency-connected architecture behind it, the main objective of this major question was to test whether the existing ML Project could actually behave like a reusable inference-based operational system when future market data entered the same workflow structure.

The first major understanding I gained is that reusable systems are not simply about automation. The real objective is to create a controlled and repeatable operational structure where future compatible data can continuously enter the same workflow without disturbing the downstream inference continuity. Because of this, schema continuity, source consistency, operational stability, and downstream compatibility became some of the most important operational requirements while building the reusable framework for the raw input layer.

To preserve this continuity, I first created a reusable extraction framework using the same source structure and schema continuity already present inside the existing ML Project data. The Yahoo Finance Python library was kept fixed as the extraction source so the future market data would remain operationally consistent with the earlier dataset continuity. The extraction workflow was also designed in a reusable and expandable way using parameters such as tickers, starting dates, and ending dates so future market data could later enter the same operational structure without rebuilding the extraction logic repeatedly.

Another important operational understanding came while merging the individual stock CSV files into one centralized dataset structure. Initially, manual merging appeared easier, but I realised that manual copy-pasting could create alignment mismatches, row inconsistencies, and synchronization problems across the centralized dataset continuity. Since the ML Project already behaves as a highly dependency-connected architecture, even small alignment issues in the source structure could later propagate throughout the downstream inference workflow.

Because of this, I implemented a reusable Python-based merging workflow using libraries such as glob and the reduce function. This helped create a scalable and reusable centralized merge structure while preserving synchronized market continuity and schema alignment across all the assets together.

Once the centralized dataset structure was prepared, another important operational step was preprocessing the data before reconnecting it with the reusable inference system. I added the required return columns using the ln function in Excel because the ML inference workflow already depended on return-based continuity instead of raw price continuity alone. Another important preprocessing step was assigning the Set_Type column as test for all the newly extracted data from November 2025 to April 2026.

This became important because the workflow was designed mainly as a reusable inference-based structure rather than a retraining-based structure. Assigning future data as train could create data leakage, downstream operational inconsistencies, and inference instability inside the workflow.

After preparing the append-ready dataset, the final operational validation started when I replaced the original raw input table inside BigQuery SQL with the updated appended dataset using the same table name already connected throughout the workflow. Preserving the same table continuity became important because the downstream views, prediction structures, operational dependencies, and reusable inference layers were already synchronized with that existing table structure.

Since the ML workflow behaves as a highly interconnected dependency-connected architecture, I also understood that validating one major downstream inference structure carefully would already provide a strong indication of whether the overall reusable continuity remained operationally stable or not.

After replacing the dataset, the downstream continuity updated successfully, and the total number of rows changed from 2467 to 2586 without generating schema issues, dependency failures, or downstream propagation errors. This became the strongest operational confirmation that my understanding of the ML Project architecture, schema continuity, downstream dependency synchronization, and reusable inference continuity was operationally correct.

Another important thing I understood from this implementation is that the ML workflow genuinely behaves like a reusable inference-based operational system when the schema continuity, source consistency, and dependency structure remain stable. The downstream reusable continuity survives because the architecture itself remains dependency-connected through views and synchronized operational structures rather than requiring repeated rebuilding of the workflow.

Overall, the answer to the third major question is that the existing ML Project successfully behaved like a reusable inference-based system when future compatible market data entered the same operational architecture. The project proved that preserving schema continuity, operational compatibility, reusable ingestion frameworks, and downstream dependency synchronization is sufficient for maintaining reusable inference continuity across the complete ML workflow without retraining the models or rebuilding the complete architecture repeatedly.

Fourth Major Question of the Project -

This is the final major question of the project and will conclude the overall reusable system analysis. The first half of the project was focused on understanding, validating, and implementing the reusable continuity of the ML workflow. Now the focus shifts toward the NLP Project system, which behaves operationally very differently from the ML architecture.

The biggest reason for this difference is that BigQuery SQL directly supports ML functions, but it does not support NLP extraction functions in the same reusable way. Because of this, the NLP workflow depends on external NLP processing through Python before the extracted outputs are uploaded back into BigQuery SQL for downstream analysis continuity.

This external extraction structure makes the NLP workflow much more modular operationally. Unlike the ML Project, which behaves as a highly centralized dependency-connected architecture, the NLP workflow is divided into more independent signal-based structures. In the ML workflow, the major operational layers mainly behave through connected models, whereas in the NLP workflow the major analytical layers behave through independent signals such as sentiment, themes, entities, and phrase-level extraction continuity.

Because the extraction process happens externally before reconnecting with BigQuery SQL, the NLP workflow does not behave with the same smooth centralized operational continuity present inside the ML architecture. Instead, each signal behaves more independently with its own extraction continuity, processing structure, and downstream analytical flow.

So the main objective of this major question is understanding how well I have understood the NLP system architecture, whether I can create reusable frameworks for the NLP extraction and ingestion process, and whether the reusable continuity of the NLP workflow can survive correctly when future compatible data is appended into the same schema structures without creating operational instability.

Another important focus of this final stage is understanding the combined operational architecture of both the ML and NLP systems together. Even though both workflows behave differently operationally, the final goal is to understand how both systems can still exist together as one reusable analytical ecosystem where the ML workflow behaves as a centralized dependency-connected inference structure while the NLP workflow behaves as a modular signal-based extraction structure.

Overall, this final major question shifts the project toward validating the more complex and externally connected NLP reusable architecture while also bringing together the complete operational understanding of both the ML and NLP systems into one broader reusable analytical framework.

Sub Question 1 -

Now the focus shifts toward building the raw-input framework for the NLP workflow. Unlike the ML Project, where historical market data was already available in a structured and reusable form, the NLP workflow originally faced a major limitation in collecting historical news continuity. Since the dataset spans nearly 10 years, retrieving older news data becomes much more difficult compared to historical stock market data, which is already available more systematically through financial data providers.

Because of this limitation, the original NLP Project was built using relatively smaller amounts of historical news data, and at that time there was no proper reusable extraction framework or structured Python-based ingestion process for collecting news continuity operationally. So one of the major objectives now is to create a reusable extraction framework for the news data itself.

While researching possible approaches for collecting future news continuity more systematically, I found that Google News RSS provides a practical reusable operational solution for recent-period news continuity. Since Google News RSS supports retrieval of recent historical news continuity for approximately the latest five to six months, it creates a usable operational structure for building a reusable NLP ingestion framework.

Because of this, I started building a reusable Python-based extraction framework around the Google News RSS structure so future news continuity can be collected, processed, and appended more systematically whenever new NLP data needs to enter the workflow later on. This significantly reduces the future scarcity problem involved in collecting recent news continuity and creates a much more reusable ingestion structure for the NLP Project going forward.

However, the situation remains different for the earnings dataset. During this stage, I was still unable to identify a stable reusable Python-based extraction framework capable of retrieving the earnings continuity in the same operational way as the news data. Because of this, the earnings extraction layer still remains partially manual and becomes one of the biggest remaining raw-input limitations inside the NLP workflow.

This also highlights another major difference between the ML and NLP systems. In the ML workflow, a reusable extraction structure for historical market continuity already existed from the beginning. But in the NLP workflow, neither the news continuity nor the earnings continuity originally had reusable ingestion frameworks. After this implementation, the news continuity at least now behaves through a reusable operational extraction structure, even though the earnings continuity still requires further operational development in the future.

For building the reusable news extraction framework, I selected Python libraries that support structured Google News RSS extraction continuity. One of the main libraries used is feedparser, which helps retrieve and process the RSS feed into structured datasets that can later be integrated into the downstream NLP workflow. I also designed the extraction structure using reusable query keywords and stock-related search continuity so the framework can remain expandable operationally if additional companies or assets need to be introduced later into the NLP workflow.

Another important operational design choice was using month-wise extraction continuity instead of retrieving all historical news together in one process. This improves operational stability during extraction and also makes the framework more reusable later because only the date continuity needs adjustment during future reruns instead of rebuilding the complete extraction structure again.

One operational challenge that emerged during implementation was that the extracted news continuity followed a different date structure compared to the existing NLP dataset continuity. To resolve this, I created another reusable Python-based append-processing workflow that standardizes the date continuity and schema structure before appending the new news continuity into the existing dataset. This becomes important because the downstream NLP workflow still depends heavily on stable formatting continuity and schema synchronization for maintaining reusable operational compatibility.

After stabilizing the news continuity, the earnings continuity still remains without a fully reusable extraction framework, which is something that will need further development later on. But at this stage, the raw-input ingestion continuity for the NLP workflow has become significantly more reusable compared to the original structure.

Another important difference from the ML workflow is that the NLP datasets do not require additional manually engineered operational columns before entering the workflow. Because of this, once the ingestion continuity is stabilized, the next major focus naturally shifts toward the Python-based NLP processing functions themselves.

Overall, this stage helped establish the reusable raw-input continuity of the NLP workflow much more systematically than before. The news extraction structure now behaves through a reusable operational framework, while the next major challenge becomes understanding whether the NLP extraction notebooks and processing functions themselves can also behave like reusable modular operational templates instead of one-time execution scripts.

Sub Question 2 -

One of the first major things I understood after building the reusable news extraction framework is that the scarcity problem of news continuity has reduced significantly for recent periods. In the original NLP Project, nearly 10 years of news continuity resulted in only around 240 news records. But for just the six-month period from November 2025 to April 2026, the reusable Google News extraction framework generated more than 3000 news records. This itself showed that news continuity becomes much easier to collect when the collection process remains active and continuous over time.

At the same time, this also helped me understand another important limitation of the NLP workflow. If the time period becomes too old, the scarcity problem can still emerge again because historical news continuity gradually becomes difficult to retrieve properly. Because of this, future reusable continuity for the NLP workflow will require continuous and prioritized news collection instead of delayed retrieval after many years.

However, the earnings continuity still does not yet have the same reusable extraction framework that now exists for both the market return continuity and the news continuity. So one of the future operational priorities of the NLP workflow will still be building a reusable extraction framework for the earnings data as well.

After stabilizing the raw-input continuity, the next major focus shifts toward the modular extraction structure of the NLP workflow itself. Unlike the ML architecture, where most downstream continuity remains centralized inside BigQuery SQL, the NLP workflow depends on separate external Python notebooks for each major NLP signal such as sentiment extraction, entity extraction, phrase extraction, and earnings sentiment analysis.

Because of this modular structure, the next important question becomes how to create reusable template continuity for these NLP extraction functions so they can later be reused operationally by mainly changing the file names or appended datasets while preserving the same downstream extraction logic and schema continuity.

One important thing I also understood here is that Python and SQL workflows behave much more stably for appended row continuity compared to highly manual spreadsheet-style workflows. Since Python and SQL process the dataset structurally instead of depending on manual row shifting, reusable continuity becomes operationally smoother. However, schema mismatches, datatype inconsistencies, or column-name differences can still create downstream operational instability, which is why preserving schema continuity remains extremely important for reusable NLP execution as well.

So now the reusable structure needs to be built around the four major NLP extraction continuities already present inside the workflow. These are the news sentiment

extraction structure, the news entity extraction structure, the news phrase extraction structure, and the earnings sentiment extraction structure.

The implementation begins with the reusable news sentiment extraction framework. To preserve operational continuity with the earlier NLP workflow, I used the same VADER sentiment analyzer that already existed inside the original project structure. This becomes important because the downstream BigQuery SQL analysis layers already depend on the same extraction continuity and schema structure. If the extraction logic changes heavily, then downstream operational compatibility can weaken because the output continuity itself changes.

Because of this, the same sentiment score continuity and magnitude-score continuity were preserved operationally inside the reusable extraction framework. This helps maintain downstream compatibility with the existing NLP workflow structure.

Similarly, the reusable entity extraction framework continues using the same spaCy-based extraction continuity already present in the original workflow. The same operational continuity is also preserved for phrase extraction because the downstream analytical structures already depend on those extraction formats. Then for earnings sentiment extraction, the same VADER-based framework was again preserved operationally so the regenerated outputs remain compatible with the downstream BigQuery SQL structures already built inside the NLP workflow.

Another important understanding from this implementation is that reusable NLP continuity depends not only on collecting future compatible data, but also on preserving stable extraction logic continuity across the Python processing layer itself. If the extraction structures, schema outputs, or operational calculation continuity change heavily, then the downstream SQL workflow can also become operationally unstable even if the raw-input continuity itself remains structurally correct.

Now, after implementing these reusable NLP extraction templates, the next operational stage becomes integrating the regenerated outputs back into the BigQuery SQL workflow and validating whether the complete NLP reusable architecture behaves correctly when future appended continuity enters the system.

Overall, this stage helped transform the NLP extraction notebooks from one-time execution structures into more reusable modular operational templates. The raw-input continuity, extraction continuity, and downstream schema continuity now behave much more systematically, while the next major challenge becomes validating whether the complete reusable NLP workflow can reconnect successfully with the regenerated extraction outputs without disturbing the downstream analytical structure.

jupyter system_news_entity_base.csv Last Checkpoint: 2 days ago

File Edit View Settings Help

Delimiter: ,

	Date	News	Entity	Entity_Type	Sallience
1	12-01-2015	ing business. (Wikipedia)	TCS	ORG	1.0
2	12-01-2015	ing business. (Wikipedia)	India	GPE	0.35
3	12-01-2015	ing business. (Wikipedia)	Reliance Industries	ORG	0.3333333333333333
4	12-01-2015	ing business. (Wikipedia)	Wikipedia	ORG	0.25
5	12-01-2015	revenue growth strategy.	Infosys	ORG	1.0
6	12-01-2015	revenue growth strategy.	quarterly	DATE	0.15
7	14-02-2015	asset pressures. (Reuters)	HDFC Bank	ORG	1.0
8	14-02-2015	asset pressures. (Reuters)	quarterly	DATE	0.15
9	14-02-2015	asset pressures. (Reuters)	around 20%	PERCENT	0.09999999999999999
10	14-02-2015	asset pressures. (Reuters)	Reuters	ORG	0.25
11	02-03-2015	and foreign investor flows.	50	CARDINAL	0.3
12	02-03-2015	and foreign investor flows.	US	GPE	0.35
13	02-03-2015	and foreign investor flows.	Indian	NORP	0.09999999999999999
14	04-03-2015	on in the Indian economy.	RBI	ORG	1.0
15	04-03-2015	on in the Indian economy.	Indian	NORP	0.15
16	24-04-2015	on under new leadership.	Infosys	ORG	1.0
17	16-05-2015	nance clients. (Wikipedia)	first	ORDINAL	0.3
18	16-05-2015	nance clients. (Wikipedia)	Shanghai	GPE	0.35
19	16-05-2015	nance clients. (Wikipedia)	Wikipedia	ORG	0.3333333333333333
20	15-12-2015	ows to emerging markets.	US Federal Reserve	ORG	1.0
21	15-12-2015	ows to emerging markets.	first	ORDINAL	0.15
22	27-12-2015	ation strategy. (Wikipedia)	Reliance Jio	PERSON	0.3

jupyter system_news_with_sentiment.csv Last Checkpoint: 2 days ago

File Edit View Settings Help

Delimiter: ,

	Date	Source	news_data	Sentiment_score	Sentiment_magnitude
1	12-01-2015	Economic Times	ing business. (Wikipedia)	0.8908	0.356
2	12-01-2015	Economic Times	revenue growth strategy.	0.7906	0.303
3	14-02-2015	Reuters	asset pressures. (Reuters)	0.8689	0.43100000000000005
4	02-03-2015	Reuters	and foreign investor flows.	0.4019	0.094
5	04-03-2015	RBI	on in the Indian economy.	0.7096	0.336
6	24-04-2015	Reuters	on under new leadership.	0.946	0.44
7	16-05-2015	Economic Times	nance clients. (Wikipedia)	0.4939	0.106
8	15-12-2015	Reuters	ows to emerging markets.	-0.2732	0.253
9	27-12-2015	Economic Times	ation strategy. (Wikipedia)	0.128	0.055
10	14-01-2016	Business Standard	hnology services market.	0.9062	0.377
11	29-02-2016	Economic Times	sting investor confidence.	0.8625	0.47000000000000003
12	29-02-2016	Economic Times	impacting equity markets.	0.4019	0.101
13	16-04-2016	Economic Times	al operations. (Wikipedia)	-0.25	0.256
14	21-04-2016	Business Standard	eteriorating asset quality.	0.5423	0.352
15	23-06-2016	Reuters	il markets and currencies.	-0.1027	0.27899999999999997
16	15-07-2016	Reuters	and consulting offerings.	0.3818	0.098
17	01-09-2016	Economic Times	ip. (The Economic Times)	-0.1531	0.056
18	05-09-2016	Reuters	ng strategies. (Wikipedia)	0.6249	0.21
19	20-10-2016	Economic Times	king systems. (Wikipedia)	0.0	0.0
20	08-11-2016	Reuters	nent systems. (Wikipedia)	0.0	0.0
21	08-11-2016	Reuters	ed selling across sectors.	-0.296	0.265
22	08-11-2016	Reuters	ts in global trade policies.	-0.296	0.109

Sub Question 3 -

Now the main task is to integrate all the regenerated Python extraction files into the BigQuery SQL workflow and test whether my understanding of the NLP Project architecture is actually operationally correct or whether there are still hidden compatibility issues inside the reusable structure. Unlike the ML workflow, where the downstream continuity behaves in a much more centralized and connected manner, the NLP workflow behaves as a modular signal-based structure. Because of this, every regenerated signal continuity needs to be checked separately again for downstream compatibility and operational synchronization.

At this stage, the process still remains highly manual and operationally fragile. The existing extraction files inside the NLP workflow need to be deleted and replaced with the regenerated extraction files using exactly the same existing table names. This becomes

important because the downstream SQL views and analysis structures remain tightly connected with those naming continuities. Because of this dependency, the replacement process itself becomes tedious and operationally sensitive even if the reusable extraction continuity survives correctly.

After uploading all the regenerated signal files along with the updated raw-input continuity, the next step was validating whether the regenerated extraction outputs remained operationally compatible with the existing downstream SQL analysis structure.

The first signal I checked was the news sentiment continuity. Inside the downstream views where the sentiment continuity connects with the Nifty return continuity, I could clearly observe the 2026 dates appearing successfully. This immediately confirmed that both the raw-input continuity and the regenerated sentiment extraction continuity were operationally synchronized with the downstream workflow.

Then I moved toward the entity extraction continuity. There I observed that the frequency of several existing entities and stock names had increased significantly. This operationally confirmed that the newly appended continuity had successfully entered the downstream entity workflow because those frequency structures could only increase if the regenerated entity extraction continuity remained compatible with the existing SQL structure.

However, the phrase extraction continuity revealed another important operational understanding. The reusable extraction framework itself was functioning correctly, but the challenge emerged from the analytical nature of the theme-building process rather than from the extraction continuity.

Inside the workflow, the Python structure extracts phrases successfully, but the actual grouping of phrases into broader themes still depends on manual analytical interpretation. The themes are formed by observing which phrases appear more frequently and then manually grouping and structuring them into downstream analytical continuity. Because of this, whenever new text continuity enters the workflow, new phrases also naturally emerge, which means new themes may need additional manual refinement or restructuring operationally.

This helped me understand that the theme signal behaves differently from the other NLP extraction structures because it remains partially interpretative and analytical by nature. Even if the extraction continuity itself survives operationally, some manual analytical refinement will still remain necessary whenever new phrase continuity enters the workflow.

At the same time, another important thing improved significantly here. Since the news source continuity has now become much more structured through the reusable Google

News framework, the phrase continuity itself also becomes operationally more stable compared to before. This reduces some randomness in future phrase generation continuity even though complete automation of the theme interpretation process still does not become fully possible.

Because of this understanding, I now clearly know that the manual refinement required in the theme signal belongs more to the analytical interpretation layer rather than to the reusable extraction architecture itself. So from the reusable-system perspective, the important thing is not whether the themes themselves remain perfectly finalized automatically, but whether the extraction continuity and downstream workflow remain operationally compatible after new data enters the structure.

Finally, I checked the earnings sentiment continuity, and the regenerated earnings sentiment extraction also remained operationally compatible with the downstream SQL analysis workflow without creating schema or synchronization issues.

Overall, this stage confirmed that there are no major compatibility problems inside the reusable Python extraction architecture itself. The reusable extraction continuity, downstream schema synchronization, and signal regeneration continuity all survived operationally after integrating the regenerated outputs back into the NLP workflow.

The only major limitation that still remains is the partially manual analytical refinement required inside the theme signal continuity, but this belongs more to the interpretative analysis layer rather than to the reusable extraction framework itself.

Another important thing I understand now is that the ML workflow and the NLP workflow survive operationally in very different ways. The ML workflow survives through centralized dependency-connected continuity, whereas the NLP workflow survives through modular signal-based continuity supported by reusable extraction layers. But even though both systems behave differently operationally, I can now finally see how both architectures can still exist together as one broader reusable analytical ecosystem where future compatible data continuously enters the workflows without requiring complete rebuilding of the overall structure again.

Query completed
Using on-demand processing quota

Query results

Job information	Results	Visualization	JSON	Execution details	Execution graph
Row	Date	Avg_sentiment	Total_magnitude	News_count	NIFTY_Returns
1	2026-04-01	0.045048275862...	5.774000000000...	29	0.015463263
2	2026-03-27	0.04790348275...	4.868	29	-0.021110314
3	2026-01-26	0.068300000000...	1.182	10	null
4	2025-11-06	0.002405555555...	2.929999999999...	18	-0.003441824
5	2025-11-18	-0.011700000000...	1.977000000000...	21	-0.003982728
6	2025-11-20	0.100515686274...	8.363999999999...	51	0.005340257
7	2026-02-17	0.141286842105...	3.903	38	0.001659286
8	2026-04-28	0.154148275862...	5.457000000000...	29	-0.004034243
9	2026-01-06	-0.025717647058...	5.193000000000...	34	-0.002731374

Results per page: 50 1 - 50 of 376

Google Cloud

My First Project

Search (/) for resources, docs, products, and more

Search

Datasets - X

nlp_entity - X

entity_imp... - X

Untitled...ery - X

Untitled query

Run

Save

Download

Share

Schedule

Open in

More

1 SELECT

2 *

3 FROM

4 "project-77c48c26-f342-4dbd-b8c.nlp_entity.entity_importance";

Query completed

Using on-demand processing quota

Query results

Chat

Save results

Open in

Job information

Results

Visualization

JSON

Execution details

Execution graph

Row	Entity	Frequency	Avg_Salience	Importance_Score	
1	India	1050	0.388205315055...	407.6155808080...	
2	Infosys	493	0.753569013812...	371.5095238095...	
3	TCS	462	0.758362536933...	350.3634920634...	
4	HDFC Bank	431	0.737690126321...	317.9444444444...	
5	Reliance	238	0.710784313725...	169.1666666666...	
6	RBI	123	0.774661246612...	95.2833333333...	
7	SBI	128	0.645507812500...	82.6250000000...	
8	Reliance Industries	90	0.798148148148...	71.8333333333...	
9	Nifty	298	0.213819050077...	63.71807692307...	
10	AI	158	0.323881856540...	51.1733333333...	

Results per page: 50 1 - 50 of 3512

Sub Question 4 -

This is the final question of the project, although at this stage it feels less like a question and more like an overall understanding of how both the ML and NLP systems behave together inside one reusable analytical framework. After completing the reusable continuity of the ML workflow and then moving into the reusable continuity of the NLP workflow, I can now see how both systems can operate together as part of one broader operational architecture instead of behaving like two completely isolated projects.

One important thing I understand now is that both systems still have significant future scalability. The ML workflow is comparatively easier to expand because of its centralized and dependency-connected architecture, while the NLP workflow is operationally more complex due to its modular extraction structure and external processing continuity. But

both systems still support future appended-data continuity and long-term reusable expansion in their own operational ways.

As the ML workflow expands further in the future through additional engineered features, OHLC-based structures, or more advanced market-return continuity, the NLP workflow can also evolve simultaneously by introducing richer signals, additional extraction layers, and more advanced downstream analytical continuity. Because of this, both workflows can continue supporting each other operationally as the broader reusable architecture grows over time.

Another important thing this project helped me understand is the complete operational continuity of the ML workflow much more deeply than when I originally built the project itself. At the time of the original ML implementation, I did not fully understand the complete downstream operational dependency structure in the way I understand it now. Through this reusable-system project, I now have a much clearer understanding of the ML architecture starting from the raw-input continuity all the way to the downstream inference and analysis continuity.

Because of this understanding, future appended-data handling and future structural expansion inside the ML workflow will now become operationally smoother and much more manageable. At the same time, I also understand more clearly that the smoothness of the ML architecture comes from its highly centralized dependency-connected continuity because BigQuery ML directly supports the downstream ML operational workflow internally. But that same centralized continuity also creates strong schema sensitivity across the complete structure, where even small schema mismatches or feature inconsistencies can propagate operationally throughout the workflow much more aggressively.

The NLP workflow behaves very differently operationally. Since BigQuery SQL does not directly support NLP extraction continuity in the same way as ML continuity, the NLP architecture depends heavily on external Python-based extraction workflows before reconnecting the processed outputs back into BigQuery SQL for downstream analysis continuity.

Because of this operational structure, the NLP workflow behaves almost opposite to the ML workflow architecturally. It remains much more modular, less dependency-connected, and operationally more manual because every major signal continuity requires separate extraction continuity, compatibility continuity, and validation continuity.

Another important thing I now understand clearly is that some level of manual analytical continuity will still remain necessary inside the NLP workflow, especially in the theme and phrase-grouping continuity where interpretation itself becomes part of the analytical

process. Even if the reusable extraction continuity survives operationally, complete automation of the interpretative theme structure still does not become fully possible operationally.

However, even with these limitations, I now have a much stronger operational understanding of the NLP architecture, its reusable extraction continuity, its modular signal behavior, and the major structural requirements necessary for preserving reusable continuity across the workflow.

When both the ML and NLP systems are finally understood together inside one reusable analytical framework, the complete operational architecture naturally becomes much more complex because both systems survive continuity in very different ways. The ML workflow survives mainly through centralized dependency-connected continuity, whereas the NLP workflow survives mainly through modular reusable extraction continuity.

But even though both architectures behave differently operationally, both still strongly support long-term reusability, scalability, and future appended-data continuity in their own ways. This overall understanding becomes extremely important because as the reusable analytical ecosystem continues expanding in the future, both the ML and NLP workflows can continue supporting future operational continuity together without requiring the complete rebuilding of the overall architecture repeatedly.

Answer of the fourth major question -

After going through all the sub questions and implementing the reusable continuity over the NLP workflow, I now understand that the Fourth Major Question was mainly about understanding how the NLP Project system behaves operationally compared to the ML Project system, whether a reusable operational framework can be built for the NLP workflow, and whether both the ML and NLP systems can survive future appended-data continuity together inside one reusable analytical architecture.

One of the biggest understandings that developed through this question is that the NLP workflow behaves completely differently compared to the ML workflow because BigQuery SQL does not directly support NLP extraction functions in the same way that BigQuery ML supports ML workflows. Because of this, the NLP workflow depends heavily on external Python extraction continuity before the extracted signals are integrated back into BigQuery SQL for downstream analysis continuity. This makes the NLP workflow much more modular and signal-based in nature compared to the centralized operational continuity of the ML workflow.

Through the implementation of the reusable extraction continuity, I was able to build a reusable operational framework for the news data extraction using Google News RSS and Python libraries like feedparser. This solved one of the biggest limitations that existed earlier in the NLP workflow, which was the scarcity and inconsistency of news data continuity. At the same time, I also understood that the earnings extraction continuity still does not have a proper reusable framework and remains one of the biggest operational limitations in the NLP workflow for now.

Another major understanding that developed was that reusable continuity in the NLP workflow depends heavily on maintaining schema continuity and downstream compatibility continuity across all the independent signals. This is why I preserved the same operational extraction logic using the same VADER and spaCy-based workflows that already existed in the original NLP workflow. By maintaining the same extraction logic and schema structure, the regenerated outputs remained operationally compatible with the existing BigQuery SQL workflow continuity.

After regenerating the reusable signal outputs, I operationally validated all the major NLP signals separately because the NLP workflow behaves modularly in nature. The sentiment continuity, entity continuity, phrase continuity, and earnings sentiment continuity all remained operationally compatible with the downstream SQL workflow. One important operational understanding that also emerged was that the phrase extraction continuity itself is reusable and stable, but the analytical process of grouping phrases into themes still remains partially manual in nature. So even if the extraction continuity survives successfully, some manual analytical refinement will always remain necessary in the theme signal whenever new phrases enter through future appended text data.

After understanding the NLP workflow operationally, I now understand that the ML and NLP workflows survive reusable continuity in opposite ways. The ML workflow survives mainly through centralized dependency-connected continuity where downstream synchronization remains highly connected. Meanwhile, the NLP workflow survives mainly through modular reusable extraction continuity where every signal behaves more independently operationally. Even though both workflows behave differently, both still strongly support future reusability, scalability, and appended-data continuity in their own operational ways.

So overall, the Fourth Major Question helped me understand much more than only how to append future data into the NLP workflow. It helped me understand the complete reusable operational architecture of both the ML and NLP systems together, how schema continuity affects both workflows differently, where manual operational limitations still exist, where modular continuity becomes important, and how both systems can continue

surviving together as one reusable analytical ecosystem when future data continuously enters the workflow.

Answer to the Problem Statement

The main objective of this project was not to build a new machine learning model or a new NLP model, but to understand whether the existing ML and NLP workflows that I had already created could move beyond static project structures and behave more like reusable analytical systems. The project mainly focused on understanding whether future market data and future text data could continuously enter the same workflow with minimal rebuilding, controlled manual effort, and stable downstream continuity.

One of the biggest understandings that led to this project was realizing the limitations of static workflows, especially the kind of structures commonly created in MS Excel, where future data usually requires repeated manual updating and restructuring. I wanted to understand whether the ML and NLP workflows could instead behave more like dynamic operational architectures where future data continuously flows through the same structure without rebuilding the complete workflow repeatedly.

This understanding became much clearer through the use of SQL views and Scrum-based workflow structures. SQL views helped create reusable downstream continuity because the analysis layers could automatically update when schema continuity remained stable. At the same time, dividing the workflows into smaller checkpoints, validation layers, and modular structures through the Scrum approach made the complete architecture more manageable and reusable operationally.

Through the ML workflow, I understood that reusable continuity becomes much smoother when the workflow behaves as a centralized dependency-connected structure. By preserving schema continuity, source continuity, feature continuity, and downstream compatibility, I was able to regenerate and append future market data into the existing ML architecture without retraining the models or rebuilding the downstream workflow. After replacing the raw-input continuity inside BigQuery SQL, the downstream analysis updated successfully without operational dependency failures, which confirmed that the ML workflow can behave like a reusable inference-based analytical system.

The NLP workflow, however, behaved very differently because BigQuery SQL does not directly support NLP extraction continuity in the same way that BigQuery ML supports ML workflows. Because of this, the NLP architecture depends heavily on external Python-based extraction continuity before reconnecting the regenerated outputs back into BigQuery SQL for downstream analysis continuity. This made the NLP workflow much

more modular, signal-based, and operationally manual compared to the centralized ML workflow.

One of the biggest operational improvements achieved in this project was building a reusable framework for news continuity extraction using Google News RSS and Python libraries such as feedparser. This significantly reduced one of the biggest limitations of the original NLP workflow, which was the scarcity and inconsistency of future news continuity. At the same time, I also understood that some operational limitations still remain, especially in the earnings extraction continuity and in the manually interpretative nature of the theme-analysis workflow.

Another major understanding from this project is that reusable systems are not simply about automation. The real objective is to create controlled operational continuity where future compatible data can repeatedly enter the same workflow without disturbing the downstream analytical structure. Reusability depends much more on preserving schema continuity, operational compatibility, downstream stability, and manageable workflow continuity rather than completely removing human involvement from the system.

Through this project, I also understood that the ML and NLP systems survive reusable continuity in very different ways. The ML workflow survives mainly through centralized dependency-connected continuity, while the NLP workflow survives mainly through modular reusable extraction continuity where every signal behaves more independently operationally. Even though both systems behave differently architecturally, both still strongly support future scalability, reusable continuity, and appended-data handling when the operational structure is understood properly.

Overall, this project successfully demonstrates that both the ML and NLP workflows can evolve beyond static project structures and behave more like reusable analytical systems. Future market data and future text data can continuously enter the same operational architecture with stable schema continuity, reusable extraction continuity, controlled manual effort, and downstream operational stability. At the same time, the project also highlights that reusable analytical systems are not fully automatic in nature, especially inside NLP workflows where subjectivity, interpretative continuity, and manual analytical refinement still remain important operational limitations. Even with these limitations, the project successfully establishes a reusable analytical ecosystem where both the ML and NLP workflows can continue evolving operationally together over time without rebuilding the complete architecture repeatedly.

Summary of the Project

This project focuses on understanding whether the existing ML and NLP workflows that I had already created could behave like reusable analytical systems instead of remaining static project structures. The main objective was to understand whether future market data and future text data could continuously enter the same workflow with minimal rebuilding, controlled manual effort, and stable downstream continuity.

The project mainly worked across two different operational architectures. The ML workflow behaved like a centralized dependency-connected structure because BigQuery ML directly supports machine learning functions inside the environment itself. In contrast, the NLP workflow behaved much more modularly because NLP extraction functions required external Python-based processing before reconnecting the outputs back into BigQuery SQL for downstream analysis continuity.

In the ML workflow, the project focused on building reusable raw-input continuity, append continuity, and downstream synchronization so future market data could enter the same inference structure without rebuilding the workflow again. The implementation successfully showed that the ML architecture can behave like a reusable inference-based system when schema continuity and operational compatibility remain stable.

In the NLP workflow, the project focused on building reusable extraction continuity for signals such as sentiment, entities, phrases, and earnings sentiment. One of the biggest improvements achieved was creating a reusable framework for future news extraction using Google News RSS and Python libraries, which significantly improved the continuity of future news-data collection.

The project also helped highlight that reusable systems are not fully automatic in nature. While reusable extraction continuity and downstream operational continuity can survive successfully, some manual analytical continuity still remains necessary, especially in the NLP theme-analysis workflow where interpretation and refinement continue evolving with new text data.

Overall, the project demonstrates that both the ML and NLP workflows can evolve beyond static analytical structures and behave more like reusable operational systems where future data can continuously enter the same architecture with stable schema continuity, reusable extraction continuity, and manageable downstream operational stability.

Key Findings

One of the biggest findings of this project is that reusable analytical systems mainly survive through stable schema continuity rather than through fixed data itself. Future market data and future text data can continuously change over time, but if the schema structure, datatypes, naming continuity, and downstream dependencies remain stable, then the reusable workflow continuity can also remain operationally stable.

Another major finding is that the ML and NLP workflows survive reusable continuity in very different architectural ways. The ML workflow behaves like a centralized dependency-connected system where downstream continuity remains highly interconnected. Because of this, even small schema inconsistencies in the raw-input layer can propagate throughout the downstream workflow. In contrast, the NLP workflow behaves much more modularly, where every signal continuity survives more independently through reusable extraction layers and separate operational validation.

The project also showed that BigQuery ML creates a much smoother reusable operational structure compared to NLP workflows because machine learning functions are directly supported inside the environment itself. Meanwhile, NLP workflows become operationally more complex because the extraction continuity depends heavily on external Python-based processing before reconnecting with the downstream SQL structure.

Another important finding is that reusable systems are not fully automatic in nature. Even when reusable extraction continuity and downstream synchronization survive successfully, some level of manual operational continuity still remains necessary. This became especially visible in the NLP theme-analysis workflow where new phrases and evolving themes continuously require interpretative refinement and analytical restructuring.

The project also highlighted the importance of reusable ingestion frameworks. Through reusable Python-based extraction structures, merge workflows, and append continuity frameworks, future data continuity became much easier to integrate operationally without rebuilding the entire workflow repeatedly. This became one of the strongest operational improvements achieved across both systems.

Overall, the project demonstrated that reusable analytical continuity is possible in both ML and NLP workflows when schema continuity, operational compatibility, reusable extraction continuity, and downstream synchronization are preserved properly across the overall architecture.

What Worked

One of the biggest things that worked successfully in this project was the reusable continuity of the ML workflow. After building the reusable extraction framework, append continuity, and schema-preserving ingestion structure, the new market data was successfully integrated into the existing ML architecture without retraining the models or rebuilding the downstream workflow. The downstream views, inference continuity, and analytical structures updated successfully after the appended data entered the system.

Another major thing that worked successfully was the reusable extraction continuity built for the NLP news workflow. The Google News RSS framework and Python-based extraction process significantly improved the continuity and availability of future news-data collection compared to the original NLP workflow, where historical news scarcity had become a major limitation.

The reusable Python extraction templates for sentiment extraction, entity extraction, phrase extraction, and earnings sentiment extraction also remained operationally compatible with the downstream BigQuery SQL workflow. By preserving the same VADER and spaCy-based extraction continuity that existed in the original NLP workflow, the regenerated outputs successfully synchronized with the downstream analytical structure without major schema or compatibility failures.

The project also successfully demonstrated that SQL views can behave like reusable operational continuity layers instead of static analytical outputs. Once the schema continuity remained stable, the downstream workflow continuity updated automatically when the new appended data entered the system. This became one of the strongest operational validations of the reusable architecture itself.

Another important thing that worked successfully was the modular checkpoint-based structure created through the Scrum approach. Dividing the workflows into smaller operational checkpoints, validation layers, and signal-based structures made the reusable system much easier to analyse, validate, and debug operationally whenever new continuity entered the workflow.

Overall, the project successfully proved that both the ML and NLP workflows can survive future appended-data continuity operationally when schema continuity, extraction continuity, and downstream compatibility continuity are preserved properly across the reusable architecture.

What Did Not Work

One of the biggest limitations that still remains in the project is that the NLP workflow is not fully reusable in the same smooth way as the ML workflow. Since BigQuery SQL does not directly support NLP extraction continuity, the NLP architecture still depends heavily on external Python-based processing and manual operational handling before reconnecting the regenerated outputs back into the SQL workflow.

Another major limitation is the earnings-data extraction continuity. Unlike the market-return continuity and the news-continuity framework that were successfully made reusable, the earnings workflow still does not have a stable reusable extraction framework for collecting future earnings continuity automatically. Because of this, the earnings layer still remains partially manual operationally.

The project also highlighted that the theme-analysis workflow inside the NLP architecture cannot become fully automatic operationally. Even when the phrase extraction continuity survives correctly, new appended text continuity naturally introduces new phrases and evolving thematic structures. Because of this, manual analytical refinement and interpretative grouping of themes still remain necessary whenever future text continuity enters the workflow.

Another limitation that became visible is the operational fragility of replacing regenerated extraction files inside the NLP workflow. Since the downstream SQL continuity depends heavily on existing naming structures and signal continuity, deleting and replacing extraction files manually still remains tedious and error-prone operationally.

The project also revealed that the ML workflow, although operationally smoother, is highly sensitive to schema mismatches because of its centralized dependency-connected continuity. Even small inconsistencies in the raw-input continuity can propagate aggressively throughout the downstream workflow and create operational instability across multiple connected structures.

Overall, the project showed that while reusable analytical continuity is operationally possible across both the ML and NLP workflows, complete automation is still not realistically achievable in every layer. Manual operational supervision, schema validation, interpretative continuity, and reusable framework maintenance still remain important parts of sustaining the overall architecture.

Final Takeaway

The biggest takeaway from this project is that reusable analytical systems are not created by removing all manual work, but by creating stable operational structures where future compatible data can continuously enter the same workflow without rebuilding the complete architecture repeatedly.

Through this project, I understood that schema continuity is the real backbone of reusable continuity. If the schema structure, datatypes, naming continuity, and downstream dependencies remain stable, then the reusable analytical workflow can also remain operationally stable even when the future data itself continuously changes over time.

Another major understanding is that different systems survive reusable continuity differently. The ML workflow survives mainly through centralized dependency-connected continuity, while the NLP workflow survives mainly through modular reusable extraction continuity. Even though both architectures behave very differently operationally, both can still function together inside one broader reusable analytical ecosystem.

The project also showed that reusable systems are not fully automatic in nature, especially in NLP workflows where interpretative continuity and manual analytical refinement still remain necessary. Because of this, the goal of reusable architecture is not complete automation, but controlled operational continuity with manageable manual intervention.

Overall, this project helped transform both the ML and NLP workflows from one-time analytical projects into reusable operational systems capable of handling future appended-data continuity with much greater structural stability, operational understanding, and long-term scalability.
